



Kafka Chaos Engineering

Playbook de Tests de Résilience (Haute Disponibilité & ACKS)

Ce document détaille les procédures pour tester la résilience d'un cluster Kafka (3 Brokers en mode KRaft) couplé au **Transactional Outbox Pattern**.

1. Entrer dans l'environnement Kafka

Pour exécuter les commandes d'administration Kafka directement depuis un conteneur :

```
docker exec -it kafka-3 bash
```

2. Vérifier l'état de santé initial (Partitions & Replicas)

```
/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-1:29092 --describe --  
topic order-events-secure
```

Vérifications attendues :

- PartitionCount: 3 et ReplicationFactor: 3
- Chaque partition possède un **Leader** et des copies réparties :
Ex: Partition 0 → Replicas: 2,3,1
- Les 3 brokers sont synchronisés dans l'**ISR (In-Sync Replicas)** :
Ex: Isr: 2,3,1

3. Chaos Test Niveau 1 : Arrêt d'un Broker

Depuis le terminal de la machine hôte, on simule une panne serveur :

```
docker stop kafka-1
```

État du cluster : 2 brokers disponibles sur 3 (**kafka-2**, **kafka-3**).

Vérification de la "Leader Election"

```
/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-2:29092 --describe --topic order-events-secure
```

Kafka a automatiquement réorganisé les leaders. L'ISR ne contient plus que 2 brokers (ex: `Isr: 2,3`).

Test applicatif (Maintenance de service)

Générer de nouveaux événements depuis l'application Spring Boot.

Résultat :  SUCCÈS.

Explication : Avec `min.insync.replicas=2` et `acks=all`, les 2 brokers restants suffisent pour confirmer l'écriture en toute sécurité.

4. Chaos Test Niveau 2 : La limite de sécurité (Block)

On coupe un deuxième broker pour passer sous le seuil de sécurité :

```
docker stop kafka-2
```

État du cluster : 1 seul broker disponible (**kafka-1** , **kafka-2** , **kafka-3** ). L'ISR tombe à 1.

Test applicatif (Protection des données)

Générer un nouvel événement depuis l'application.

Résultat :  ÉCHEC KAFKA (Refus d'écriture).

- Kafka lève une `NotEnoughReplicasException` car `ISR (1) < min.insync.replicas (2)`.
- **L'Outbox Pattern sauve le système :** L'erreur est interceptée par le Poller. L'événement reste en base de données avec `processed = false`. Aucune donnée n'est perdue.

5. La Résurrection (Auto-Healing)

On rallume les serveurs pour simuler la fin de la panne réseau :

```
docker start kafka-1 kafka-2
```

- Les brokers se reconnectent et rattrapent leur retard (Catch-up) pour réintégrer l'ISR.
- **Auto-réparation applicative :** Dès que Kafka est prêt, le Poller Outbox expédie l'événement bloqué. Le système se répare tout seul sans intervention humaine !

Résumé du Flux de Résilience

Cluster OK (3 Brokers, ISR=3) → Écriture 

↓

Panne Kafka-1 (2 Brokers, ISR=2) → Écriture  (Tolérance aux pannes)

↓

Panne Kafka-2 (1 Broker, ISR=1) → Écriture  (Protection des données)

↓

Sauvegarde Outbox BDD → processed = false

↓

Redémarrage Kafka-1 & 2 → Auto-rattrapage ISR → Poller envoie → BINGO 